

## Unit IV Activity II

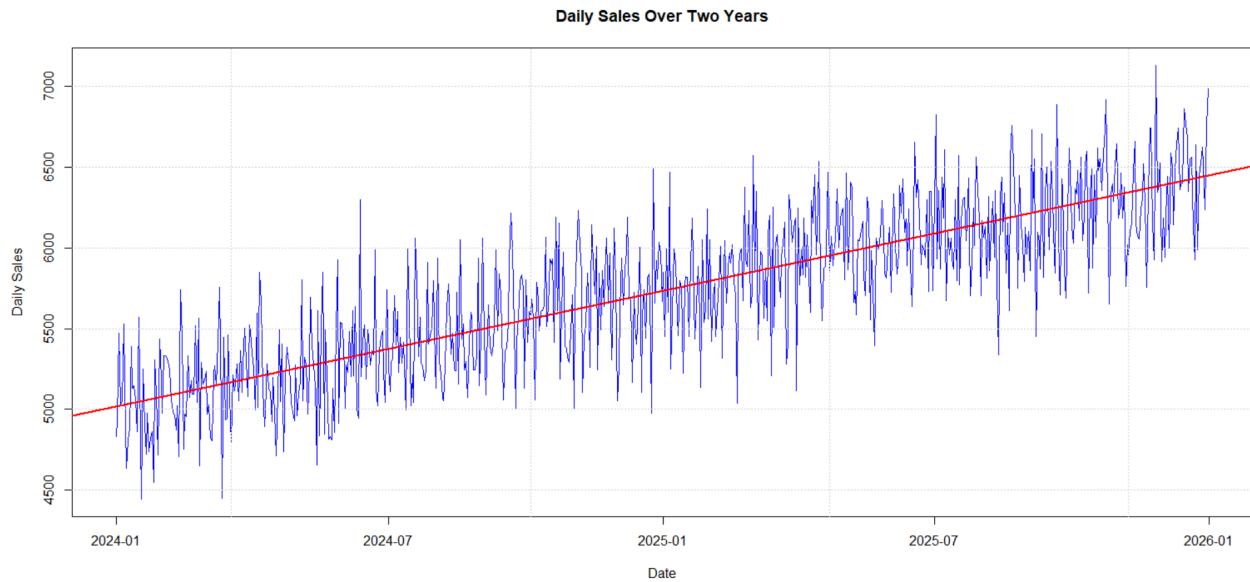
Date:19-08-2026

### 1. Daily Sales Time Series Analysis

#### R Program Code:

```
set.seed(123)
dates <- seq(
  as.Date("2024-01-01"),
  as.Date("2025-12-31"),
  by = "day"
)
sales <- 5000 +
  (1:length(dates)) * 2 +
  rnorm(length(dates), mean = 0, sd = 300)
sales_data <- data.frame(
  Date = dates,
  Sales = sales
)
plot(
  sales_data$Date,
  sales_data$Sales,
  type = "l",
  main = "Daily Sales Over Two Years",
  xlab = "Date",
  ylab = "Daily Sales",
  col = "blue"
)
grid()
trend <- lm(Sales ~ Date, data = sales_data)
abline(
  trend,
  col = "red",
  lwd = 2
)
```

#### Output:



## 2. Server Temperature Time Series

### Python Program Code:

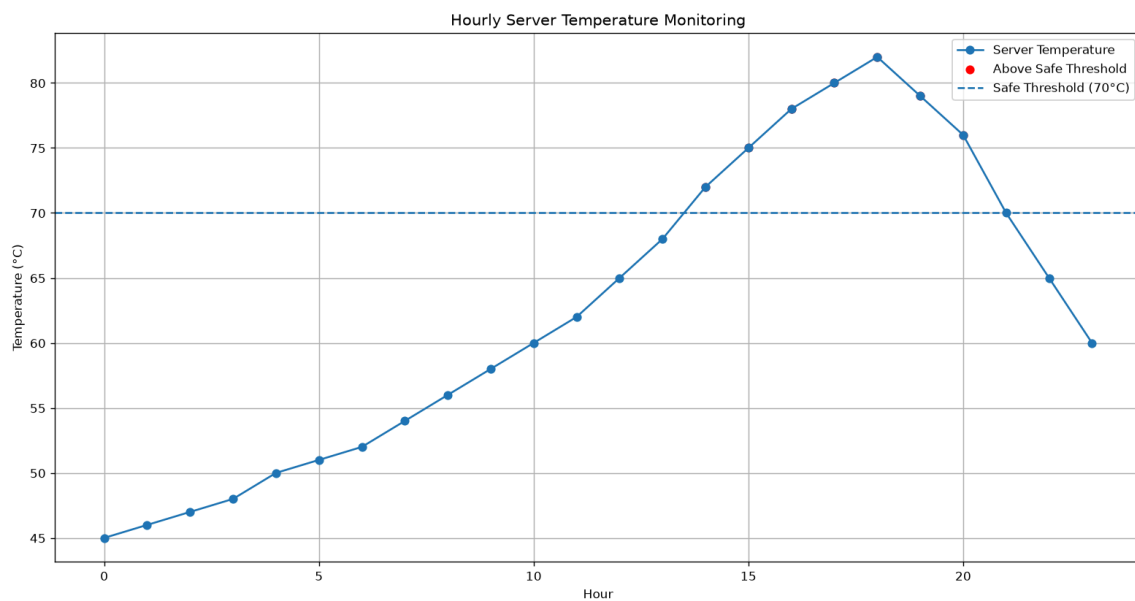
```
import numpy as np
import matplotlib.pyplot as plt
hours = np.arange(24)
temperature = [
    45, 46, 47, 48, 50, 51,
    52, 54, 56, 58, 60, 62,
    65, 68, 72, 75, 78, 80,
    82, 79, 76, 70, 65, 60
]
safe_threshold = 70
# Plot temperature
plt.figure(figsize=(10, 6))
plt.plot(
    hours,
    temperature,
    marker="o",
    label="Server Temperature"
)
high_temp = np.array(temperature) > safe_threshold
plt.scatter(
    hours[high_temp],
```

```

np.array(temperature)[high_temp],
color="red",
label="Above Safe Threshold"
)
plt.axhline(
    y=safe_threshold,
    linestyle="--",
    label="Safe Threshold (70°C)"
)
plt.xlabel("Hour")
plt.ylabel("Temperature (°C)")
plt.title("Hourly Server Temperature Monitoring")
plt.grid(True)
plt.legend()
plt.show()

```

### Output:



### 3. Comparison of Three Tech Stock Prices

#### R Program Code:

```

set.seed(123)
days <- 1:30
Stock_A <- 100 + cumsum(rnorm(30, 0.5, 2))

```

```

Stock_B <- 100 + cumsum(rnorm(30, 0.5, 1))
Stock_C <- 100 + cumsum(rnorm(30, 0.5, 3))
plot(
  days,
  Stock_A,
  type = "l",
  lwd = 2,
  col = "blue",
  xlab = "Day",
  ylab = "Closing Price",
  main = "Daily Closing Prices of Three Tech Stocks",
  ylim = range(c(Stock_A, Stock_B, Stock_C))
)
lines(
  days,
  Stock_B,
  col = "red",
  lwd = 2
)
lines(
  days,
  Stock_C,
  col = "green",
  lwd = 2
)
legend(
  "topleft",
  legend = c("Stock A", "Stock B", "Stock C"),
  col = c("blue", "red", "green"),
  lwd = 2
)
grid()

```

**Output:**

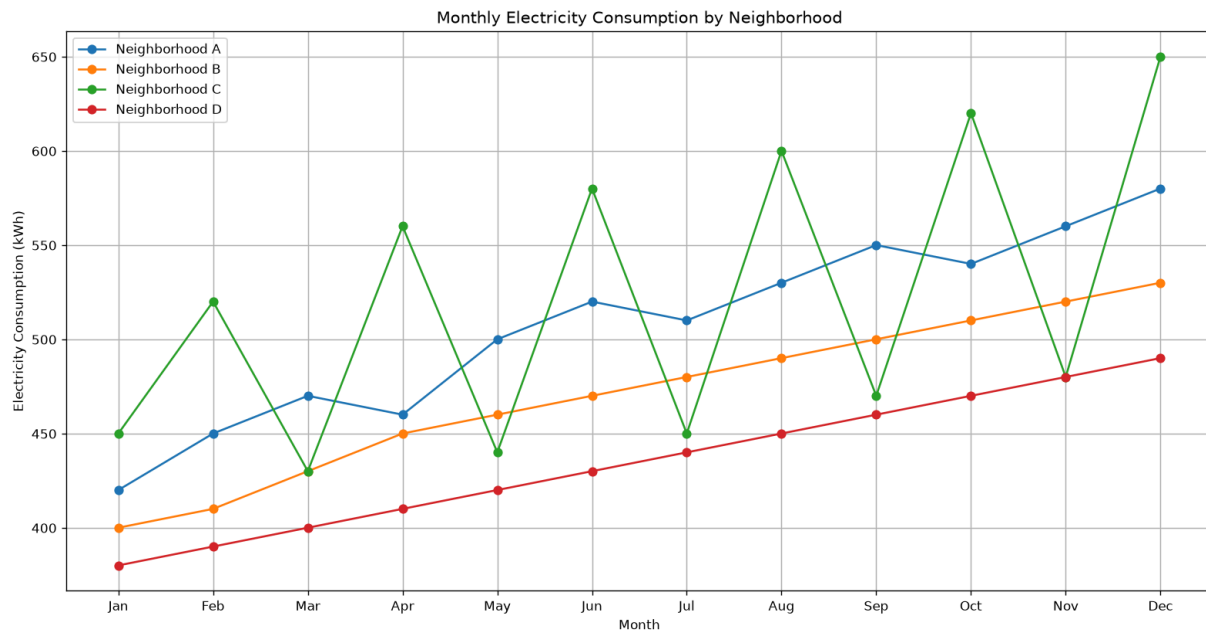


#### 4. Monthly Electricity Consumption

##### Python Program Code:

```
import matplotlib.pyplot as plt
months = [
    "Jan", "Feb", "Mar", "Apr", "May", "Jun",
    "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"
]
neighborhood_A = [420, 450, 470, 460, 500, 520, 510, 530, 550, 540, 560, 580]
neighborhood_B = [400, 410, 430, 450, 460, 470, 480, 490, 500, 510, 520, 530]
neighborhood_C = [450, 520, 430, 560, 440, 580, 450, 600, 470, 620, 480, 650]
neighborhood_D = [380, 390, 400, 410, 420, 430, 440, 450, 460, 470, 480, 490]
plt.figure(figsize=(10, 6))
plt.plot(months, neighborhood_A, marker="o", label="Neighborhood A")
plt.plot(months, neighborhood_B, marker="o", label="Neighborhood B")
plt.plot(months, neighborhood_C, marker="o", label="Neighborhood C")
plt.plot(months, neighborhood_D, marker="o", label="Neighborhood D")
plt.xlabel("Month")
plt.ylabel("Electricity Consumption (kWh)")
plt.title("Monthly Electricity Consumption by Neighborhood")
plt.grid(True)
plt.legend()
plt.show()
```

## Output:



## 5. Dose-Response Curve Analysis

### R Program Code:

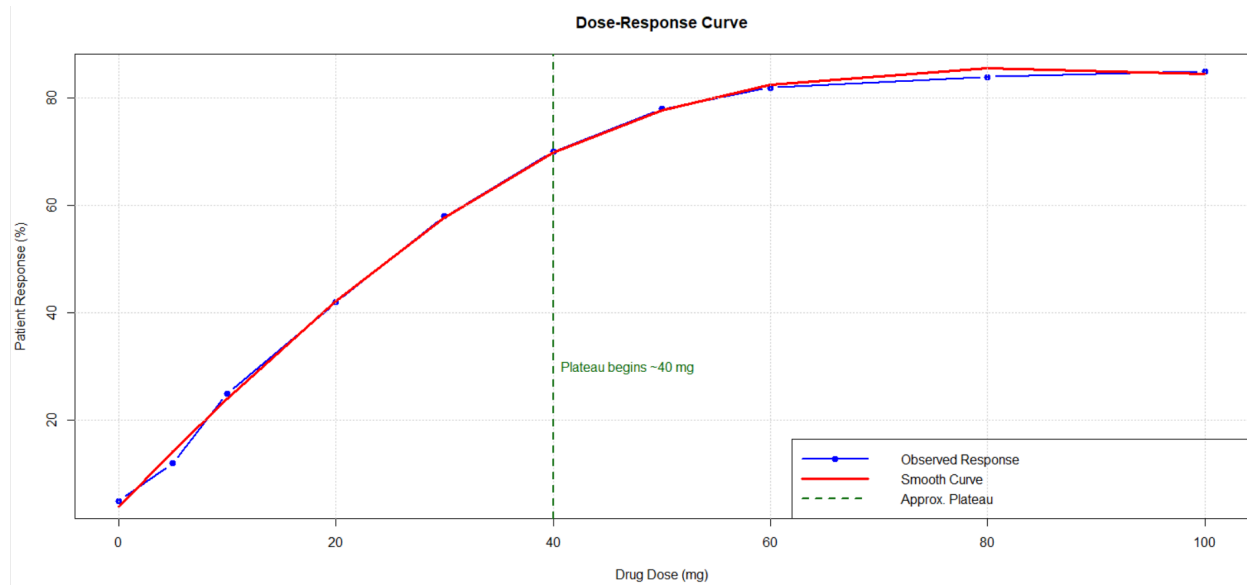
```
dose <- c(0, 5, 10, 20, 30, 40, 50, 60, 80, 100)
response <- c(5, 12, 25, 42, 58, 70, 78, 82, 84, 85)
data <- data.frame(dose, response)
plot(
  data$dose,
  data$response,
  type = "b",
  pch = 19,
  col = "blue",
  lwd = 2,
  main = "Dose-Response Curve",
  xlab = "Drug Dose (mg)",
  ylab = "Patient Response (%)"
)
smooth <- loess(response ~ dose, data = data)
lines(
```

```

data$dose,
predict(smooth),
col = "red",
lwd = 3
)
grid()
abline(
  v = 40,
  col = "darkgreen",
  lty = 2,
  lwd = 2
)
text(
  40,
  30,
  "Plateau begins ~40 mg",
  pos = 4,
  col = "darkgreen"
)
legend(
  "bottomright",
  legend = c("Observed Response", "Smooth Curve", "Approx. Plateau"),
  col = c("blue", "red", "darkgreen"),
  lwd = c(2, 3, 2),
  pch = c(19, NA, NA),
  lty = c(1, 1, 2)
)

```

**Output:**



## 6. Fertilizer Dose–Response Curve

### Python Program Code:

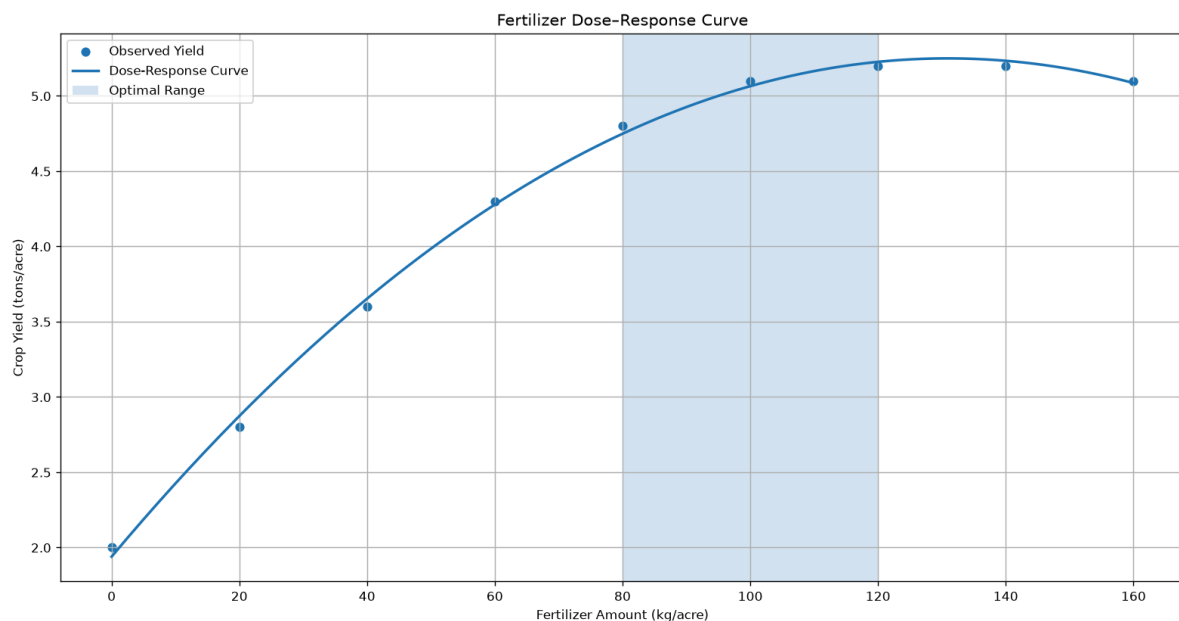
```
import numpy as np
import matplotlib.pyplot as plt
fertilizer = np.array([0, 20, 40, 60, 80, 100, 120, 140, 160])
yield_data = np.array([2.0, 2.8, 3.6, 4.3, 4.8, 5.1, 5.2, 5.2, 5.1])
coefficients = np.polyfit(fertilizer, yield_data, 2)
curve = np.poly1d(coefficients)
x_smooth = np.linspace(0, 160, 200)
y_smooth = curve(x_smooth)
plt.figure(figsize=(9, 6))
plt.scatter(
    fertilizer,
    yield_data,
    label="Observed Yield"
)
plt.plot(
    x_smooth,
    y_smooth,
    linewidth=2,
    label="Dose-Response Curve"
)
plt.axvspan(
    80,
```

```

120,
alpha=0.2,
label="Optimal Range"
)
plt.xlabel("Fertilizer Amount (kg/acre)")
plt.ylabel("Crop Yield (tons/acre)")
plt.title("Fertilizer Dose–Response Curve")
plt.grid(True)
plt.legend()
plt.show()

```

**Output:**



## 7. STL Decomposition of Monthly Airline Ticket Sales — R

**R Program Code:**

```

set.seed(123)
sales <- c(
  120, 135, 150, 145, 160, 180, 210, 200, 170, 155, 180, 230,
  115, 130, 145, 140, 155, 175, 205, 195, 165, 150, 175, 225,
  110, 125, 140, 135, 150, 170, 200, 190, 160, 145, 170, 220
)

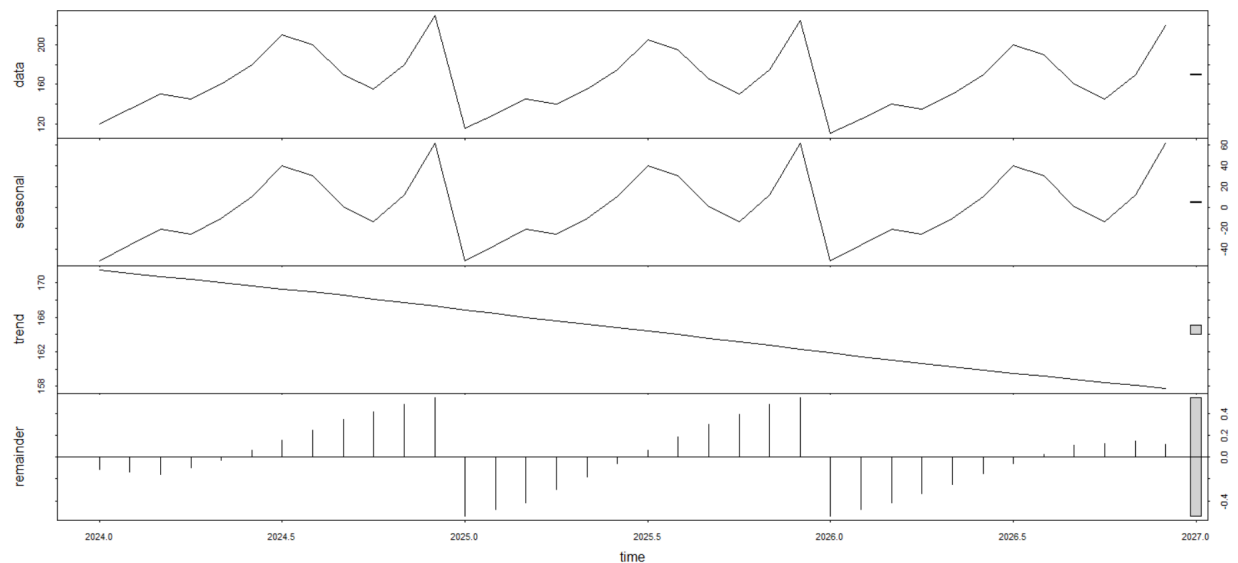
```

```

ticket_ts <- ts(
  sales,
  start = c(2024, 1),
  frequency = 12
)
plot(
  ticket_ts,
  main = "Monthly Airline Ticket Sales",
  xlab = "Year",
  ylab = "Ticket Sales",
  col = "blue"
)
stl_result <- stl(
  ticket_ts,
  s.window = "periodic"
)
plot(stl_result)

```

### Output:



## 8. STL Decomposition of Weekly User Activity — Python

### Python Program Code:

```

import numpy as np
import matplotlib.pyplot as plt

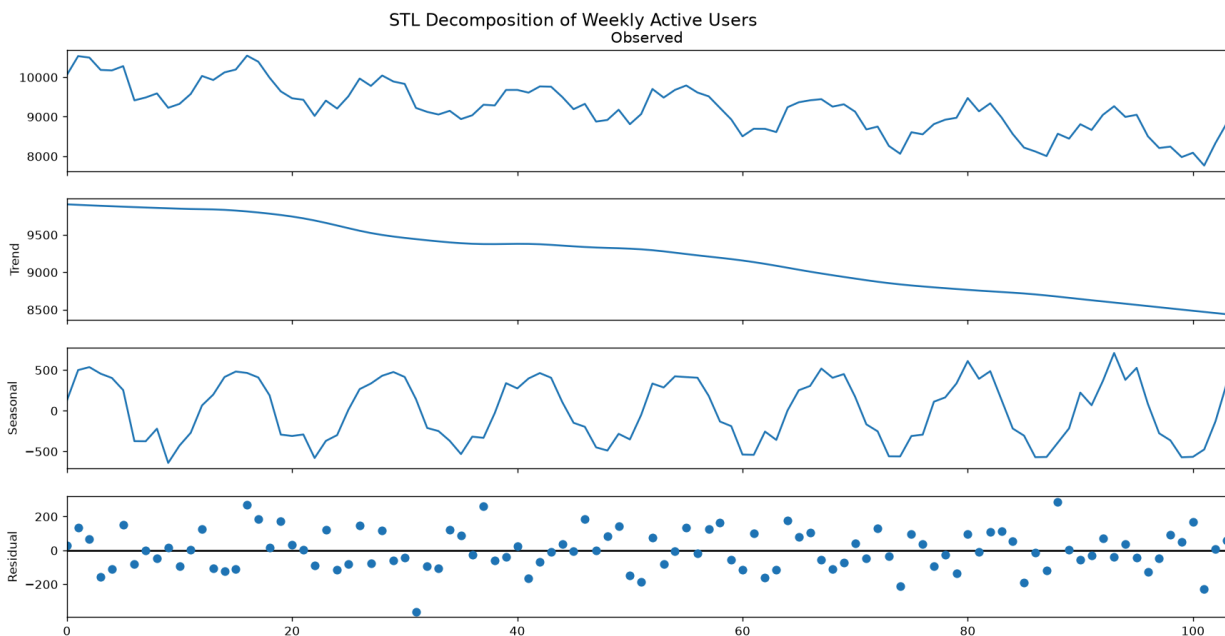
```

```

from statsmodels.tsa.seasonal import STL
np.random.seed(123)
weeks = np.arange(1, 105)
trend = 10000 - (weeks * 15)
seasonal = 500 * np.sin(2 * np.pi * weeks / 13)
noise = np.random.normal(0, 150, len(weeks))
users = trend + seasonal + noise
user_ts = users
stl = STL(
    user_ts,
    period=13
)
result = stl.fit()
result.plot()
plt.suptitle(
    "STL Decomposition of Weekly Active Users",
    fontsize=14
)
plt.show()

```

## Output:



## 9. Moving Average of Stock Prices — R

### **R Program Code:**

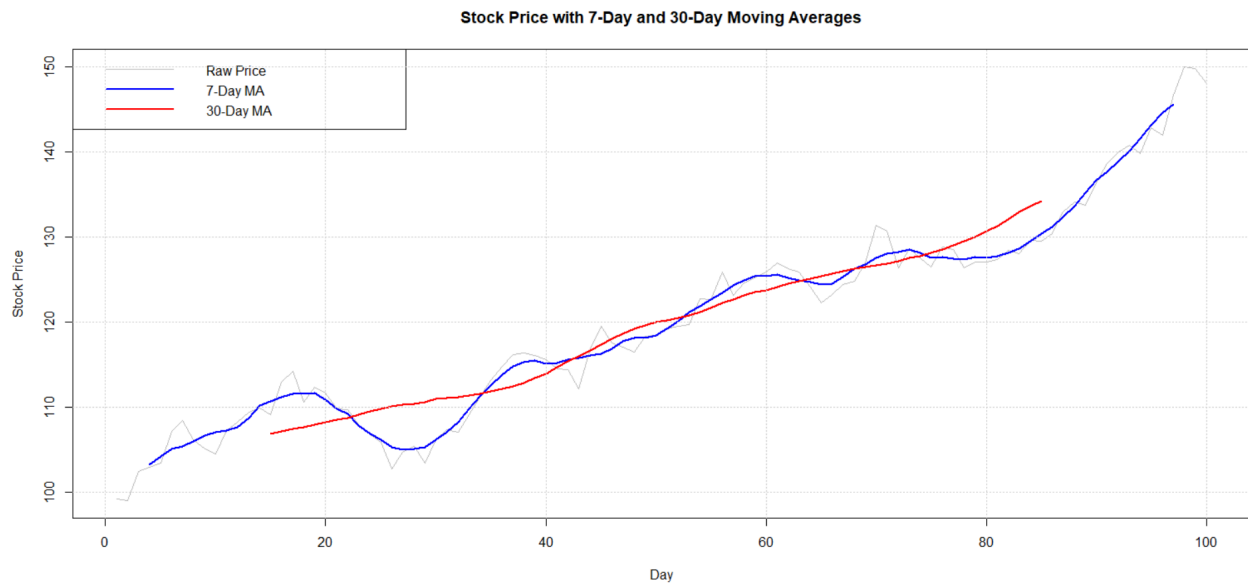
```
set.seed(123)
days <- 1:100
price <- 100 + cumsum(rnorm(100, mean = 0.3, sd = 2))
MA_7 <- stats::filter(
  price,
  rep(1/7, 7),
  sides = 2
)
MA_30 <- stats::filter(
  price,
  rep(1/30, 30),
  sides = 2
)
plot(
  days,
  price,
  type = "l",
  col = "gray",
  lwd = 1,
  main = "Stock Price with 7-Day and 30-Day Moving Averages",
  xlab = "Day",
  ylab = "Stock Price"
)
lines(
  days,
  MA_7,
  col = "blue",
  lwd = 2
)
lines(
  days,
  MA_30,
  col = "red",
  lwd = 2
)
legend(
  "topleft",
  legend = c("Raw Price", "7-Day MA", "30-Day MA"),
  col = c("gray", "blue", "red"),
```

```

lwd = c(1, 2, 2)
)
grid()

```

**Output:**



## 10. Detrending Quarterly GDP — Python

### Python Program Code:

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import detrend
quarters = np.arange(1, 25)
GDP = np.array([
    500, 505, 510, 518,
    525, 532, 540, 548,
    555, 563, 570, 578,
    585, 592, 600, 608,
    615, 623, 630, 638,
    645, 653, 660, 668
])
GDP_detrended = detrend(GDP)
plt.figure(figsize=(10, 5))
plt.plot(

```

```

    quarters,
    GDP,
    marker="o"
)
plt.xlabel("Quarter")
plt.ylabel("GDP")
plt.title("Quarterly GDP")
plt.grid(True)
plt.show()
plt.figure(figsize=(10, 5))
plt.plot(
    quarters,
    GDP_detrended,
    marker="o"
)
plt.axhline(
    y=0,
    linestyle="--"
)
plt.xlabel("Quarter")
plt.ylabel("Detrended GDP")
plt.title("Detrended Quarterly GDP")
plt.grid(True)
plt.show()

```

### Output:

